

Java Fun Games

10 Command-Line Assignments for Beginners

Arrays • Loops • If-Else • Switch • Strings • Functions — nothing else needed

A note before you start

This set of programs is voluntary. It is not an assignment, there is no deadline and no marks. We will work through these together in class later as well, since we have holidays ahead. It is here for those of you who have some free time during the festival season and feel like keeping the coding habit alive — even one or two hours a day is enough to finish a game or two.

Each game is a small command-line program that takes a name, a date of birth or a few answers as input and prints back something funny about the person. They are meant to be built, then run on your friends and family. That is the fun part — a program that makes your cousin laugh teaches you more than one that only prints 'Hello World'.

How to attempt these programs

- **Start with games 1 and 3.** Lucky Number and Name Vibe Meter are the easiest and use everything you need for the rest. FLAMES (5) and Days on Earth (2) are the most satisfying to finish; leave them for when you are warmed up.
- **Read the whole assignment first,** including the sample run and the hints, before writing a line. Trace the sample by hand on paper. If your hand trace matches the sample output, your code will too.
- **Make the numbers match, then make it funny.** Your program must produce the same numbers as the sample for the same input. The jokes, the roasts and the messages can be entirely your own — in fact they should be.
- **Do not crash on silly input.** Type letters where a number is expected, type 30 February, press Enter on an empty name. A finished program politely asks again. The reusable input helper on the next page does this for you.
- **Write it the clean way.** Follow the coding guidelines in the next section — proper names, small methods, constants, comments. It takes ten extra minutes and it is the habit that gets you through a code review at your first job.
- **Try the 'Level up' ideas** at the end of each assignment once the basic version works.
- **Share it.** Run your program on a friend, a sibling or a parent and see their reaction. Post a screenshot of the funniest output in the group.

Bonus: build your own game

Once you have finished two or three of these, invent your own. Use the same ingredients — a name, a date, a few yes/no answers, some arrays of messages — and think about what would make the people around you laugh. Some starting points: a 'how filmy is your name' meter, a biryani compatibility checker for two friends, a 'what is your cricket batting position' predictor from a date of birth, a 'which South Indian breakfast are you' quiz, or a roast generator for your college. Write it with the same care as the ten here, and share it in the group. The best original games will be demoed in class when we are back.

What you need to know

Every program in this set uses only the Java you already know: variables and data types, if-else, switch, for / while / do-while loops, nested loops, arrays, Strings and static methods. No ArrayList, no HashMap, no classes beyond the one holding main, no exceptions, no external libraries. The only import you need is `java.util.Scanner`.

Each assignment takes input from the user on the command line and prints back something funny about them — their name, their date of birth, how long they have been on Earth, and so on. Make the output as entertaining as you like; the learning is in the logic and the code quality, the laughs are a bonus.

The core logic of each game fits in 25 to 50 lines. A finished program will be longer than that — usually 120 to 200 lines — because a professional program also validates its input, names its constants, splits work into small methods and explains itself with comments. That extra length is the point of these assignments, not a problem to be avoided.

Coding guidelines to follow

Correct output is half the job. The other half is writing code another person can read without effort. Follow these rules in every program:

- **Names say what things are.** Variables are nouns (`birthYear`, `remainingLetters`, `hasDigit`), methods are verbs (`calculateLuckyNumber`, `readName`, `isLeapYear`). No single-letter names except loop counters like `i` and `j`. No abbreviations a stranger would have to guess.
- **No magic numbers.** Every number with a meaning gets a named constant: `private static final int MAX_ATTEMPTS = 7`; The reader should never have to ask 'why 101?'
- **One method, one job.** `main` reads inputs, calls a few well-named methods and prints. Calculation, validation and output live in separate methods. A method longer than about 25 lines is a sign it is doing two things.
- **One idea per line.** Give intermediate results a name instead of nesting calls: `int dayDigit = reduceToSingleDigit(day)`; is better than one long expression. Prefer a plain if-else over a clever ternary or a chain of `&&` and `||` nobody can read.
- **Validate every input, never crash.** Read each value as a line of text with `nextLine()`, check it (is it all digits? is it in range? is it non-empty?), and if it is bad, print a specific message and ask again. A program that throws an exception on 'abc' is not finished.
- **Comment the why, not the what.** A short comment above each method saying what it does and one example is worth more than a comment on every line. Comment the tricky spots: a break inside a nested loop, a % that wraps around, a formula step.
- **Consistent layout.** Four-space indentation, braces on every if and loop body even when it is one line, a blank line between methods, constants at the top of the class, input helpers together, game logic together, output together.
- **Same numbers as the sample.** Your program must produce the sample output shown for the sample input. The jokes can be yours; the arithmetic cannot.

How to compile and run: save the file as `ClassName.java` (the class name must match the file name), then in a terminal run `javac ClassName.java` followed by `java ClassName`.

The input pattern you will reuse everywhere

Eight of the ten programs need to read a whole number safely. Write this pair of methods once, understand it, and reuse it. It reads a line, checks that every character is a digit, converts it, checks the range, and loops until all of that is true — without ever throwing an exception.

```
private static int readNumberInRange(Scanner scanner, String prompt, int min, int max)
{
    while (true) {
        System.out.print(prompt);
        String input = scanner.nextLine().trim();
```

```

        if (!isWholeNumber(input)) {
            System.out.println("'" + input + "' is not a number. Please enter digits
only.");
            continue;
        }

        int value = Integer.parseInt(input);
        if (value < min || value > max) {
            System.out.println("Please enter a value between " + min + " and " + max +
".");
            continue;
        }

        return value;
    }
}

private static boolean isWholeNumber(String text) {
    if (text.isEmpty()) {
        return false;
    }
    for (int i = 0; i < text.length(); i++) {
        char character = text.charAt(i);
        if (character < '0' || character > '9') {
            return false;
        }
    }
    return true;
}
}

```

The ten assignments

- **1. Lucky Number Personality Predictor** — Numerology, but make it roast.
- **2. How Long Have You Been Loading? (Days on Earth)** — Your age in days, hours, heartbeats and cups of chai.
- **3. Name Vibe Meter** — Is your name a Bollywood song or a bank form?
- **4. Mirror Name & Secret Spy Code** — Meet your evil twin from the mirror universe.
- **5. FLAMES Calculator** — The last-bench relationship algorithm, finally in Java.
- **6. Zodiac Roast Machine** — Your star sign, but honest.
- **7. Which Day Were You Born?** — Monday blues since Day 1. Literally.
- **8. Which Chai Are You? (Personality Quiz)** — Filter coffee, masala chai, cutting chai or cold coffee — the code decides.
- **9. Guess the Number (With Attitude)** — Seven attempts. The computer will not be kind.
- **10. Password Roaster** — Your grandmother's phone lock is stronger than this.

Assignment 1: Lucky Number Personality Predictor

Numerology, but make it roast.

Concepts you will use

Data types	int for day, month, year; String for the name; constants for the valid ranges
Loops	while loop to keep adding digits until a single digit remains; while loop for re-asking on bad input
Decisions	if for validation; switch on the final number (cases 1 to 9) with a default
Functions	reduceToSingleDigit, sumOfDigits, calculateLuckyNumber, getPersonalityReading, plus input helpers

The task

Ask the user for their name and date of birth (day, month and year, each asked separately). Reduce the day to a single digit, the month to a single digit, and the year to a single digit by repeatedly adding the digits together. Add those three results and reduce again. The final single digit (1 to 9) is the user's lucky number.

Print the lucky number and then a one-line 'personality reading' for that number. Make each of the nine readings funny and relatable — the boss who cancels plans, the planner whose timetable is never followed, the hustler with three startup ideas and zero lines of code.

Input rules: the name must not be empty; day must be 1-31, month 1-12, year 1900-2026. If the user types letters where a number is expected, or a number outside the range, print a helpful message and ask the same question again.

Step by step

- Write `sumOfDigits(int number)` that adds up the digits using `% 10` and `/ 10`.
- Write `reduceToSingleDigit(int number)` that calls `sumOfDigits` repeatedly while the number is greater than 9.
- Write `calculateLuckyNumber(day, month, year)` that reduces each part, adds them, and reduces once more.
- Write `getPersonalityReading(int luckyNumber)` that returns a String from a switch.
- Write `readName` and `readNumberInRange` helpers that loop until the input is valid. `main` should only call these methods and print.

Sample run

(What the user types is shown after the prompt. Your jokes may differ; the numbers must not.)

Sample 1

```
Enter your name: Sanat
Enter your birth day (1-31): 12
Enter your birth month (1-12): 8
Enter your birth year (1900-2026): 2009

Hey Sanat, your lucky number is 4!
The Planner. You have a timetable. It is not being followed.
```

Sample 2

```
Enter your name:
Name cannot be empty. Please try again.
```

```
Enter your name: Priya
Enter your birth day (1-31): 29
Enter your birth month (1-12): abc
'abc' is not a number. Please enter digits only.
Enter your birth month (1-12): 11
Enter your birth year (1900-2026): 2005

Hey Priya, your lucky number is 2!
The Peacemaker. You say 'anything is fine' and then judge the biryani.
```

Hints

- $\text{number} \% 10$ gives the last digit; $\text{number} / 10$ removes it. Loop while $\text{number} > 0$.
- Why does 2009 become 2? $2+0+0+9 = 11$, then $1+1 = 2$. `reduceToSingleDigit` calls `sumOfDigits` twice.
- Test with 1999 ($1+9+9+9 = 28$, $2+8 = 10$, $1+0 = 1$). Three rounds!
- Read every input as a line of text with `nextLine()`, check that it contains only digits, and only then convert with `Integer.parseInt`. This avoids the classic Scanner mix-up between `nextInt` and `nextLine`.

Level up (optional)

- Also print the lucky number of the user's crush and declare them 'compatible' if the two numbers add up to an even number.
- Let the user run it again for a friend using a do-while loop with a 'Try another? (y/n)' prompt.

Assignment 2: How Long Have You Been Loading? (Days on Earth)

Your age in days, hours, heartbeats and cups of chai.

Concepts you will use

Data types	long for big totals (heartbeats cross 2 billion — int overflows!); int[] to carry a date
Arrays	DAYS_IN_MONTH lookup table; a date returned as {day, month, year}
Loops	for loops over years and over months; while loop for input validation
Decisions	if-else for leap years; guard against a 'today' that is before the birth date
Functions	isLeapYear, getDaysInMonth, countDaysFromYearOne, calculateDaysBetween, printLifeStatistics

The task

Ask for the user's date of birth and today's date. Work out exactly how many days the user has been alive, without using any Date or Calendar class — count the days yourself, using an array of month lengths and a leap-year check.

Then print a set of ridiculous but correct statistics: hours alive, weeks alive, approximate heartbeats (about 70 per minute), full days spent sleeping (a third of life), and cups of chai consumed (two a day, minimum). Finish by telling them how many days remain until their next 1000-day milestone.

Input rules: read the year first, then the month, then the day, so that the day can be limited to the real length of that month (30 February must be rejected; 29 February accepted only in a leap year). If today's date is earlier than the birth date, say so politely and stop.

Step by step

- isLeapYear(year): divisible by 400 → true; divisible by 100 → false; otherwise divisible by 4.
- getDaysInMonth(month, year): look up DAYS_IN_MONTH and add 1 for February in a leap year.
- countDaysFromYearOne(day, month, year): loop over all full years before this year, then all full months before this month, then add the day.
- calculateDaysBetween(fromDate, toDate) = countDaysFromYearOne(toDate) - countDaysFromYearOne(fromDate).
- printLifeStatistics(daysAlive): derive everything with named constants (HOURS_PER_DAY, HEARTBEATS_PER_MINUTE ...).

Sample run

(What the user types is shown after the prompt. Your jokes may differ; the numbers must not.)

Sample 1

```
--- Your date of birth ---
Year (1900-2100): 1982
Month (1-12): 4
Day (1-30): 4
--- Today's date ---
Year (1900-2100): 2026
Month (1-12): 9
Day (1-30): 7

You have been loading on Earth for 16227 days.
That is 389448 hours, and 2318 weeks.
Your heart has beaten about 1635681600 times. Mostly for exams.
You spent about 5409 full days sleeping. Respect.
```

```
Approx. chai consumed: 32454 cups. Doctors are worried.  
Next 1000-day milestone in 773 days. Set a reminder.
```

Sample 2

```
--- Your date of birth ---  
Year (1900-2100): 2024  
Month (1-12): 2  
Day (1-29): 30  
Please enter a value between 1 and 29.  
Day (1-29): 29  
--- Today's date ---  
Year (1900-2100): 2024  
Month (1-12): 3  
Day (1-31): 1  
  
You have been loading on Earth for 1 days.  
That is 24 hours, and 0 weeks.  
Your heart has beaten about 100800 times. Mostly for exams.  
You spent about 0 full days sleeping. Respect.  
Approx. chai consumed: 2 cups. Doctors are worried.  
Next 1000-day milestone in 999 days. Set a reminder.
```

Hints

- Use long for the totals. $16227 \times 24 \times 60 \times 70$ is bigger than the largest int (about 2.1 billion).
- The simplest correct way to find days between two dates is to count days from a fixed starting point to each date, then subtract.
- Test with birthday = today: the answer must be 0. Test with 29 Feb 2024 vs 1 Mar 2024: the answer must be 1.
- Return the three parts of a date as an int[] {day, month, year} so a single method can read a whole date.

Level up (optional)

- Print the user's age on other planets: Mercury year = 88 Earth days, Mars = 687, Jupiter = 4333. Use an array for the planet names and another for the day counts.
- Print how many days until their next birthday.
- Fix the grammar: '1 days' should read '1 day'.

Assignment 3: Name Vibe Meter

Is your name a Bollywood song or a bank form?

Concepts you will use

Strings	toUpperCase(), length(), charAt(i), trim(), isEmpty()
Characters	range checks like <code>c >= 'A' && c <= 'Z'</code> ; <code>c - 'A' + 1</code> turns a letter into its alphabet position
Loops	for loop over every character, once per statistic; while loop for validation
Decisions	if / else-if chains comparing counters; small boolean helper methods
Functions	isLetter, isVowel, countVowels, countConsonants, calculateLetterPower, getVowelVerdict, getScoreVerdict

The task

Read the user's full name. Count vowels and consonants (ignore spaces). Also compute the 'letter power' — the sum of every letter's position in the alphabet (A=1, B=2 ... Z=26). Turn that into a Vibe Score from 0 to 100 using % 101.

Print the counts, the score, then two funny verdicts: one based on vowels versus consonants (MUSICAL, BALANCED, HEAVY METAL, CLASSIC) and one based on the score (main character energy, reliable side character, or the friend at the edge of every group photo).

Input rules: the name must not be empty and may contain only letters and spaces. Anything else (digits, symbols) is rejected with a message and the question is asked again.

Sample run

(What the user types is shown after the prompt. Your jokes may differ; the numbers must not.)

Sample 1

```
Enter your full name: Priya123
Please use letters and spaces only (no digits or symbols).
Enter your full name: Priya Sharma

Name: PRIYA SHARMA
Vowels: 4  Consonants: 7
Letter power (A=1..Z=26 total): 129
Vibe Score: 28/100
Vibe: CLASSIC. Works in a movie, works on a bank form.
Rating: The friend who is always in the group photo, at the edge.
```

Sample 2

```
Enter your full name: Aaeesha

Name: AAEESSHA
Vowels: 5  Consonants: 2
Letter power (A=1..Z=26 total): 40
Vibe Score: 40/100
Vibe: MUSICAL. Your name is 60% vowels, like a Bollywood song.
Rating: The friend who is always in the group photo, at the edge.
```

Hints

- Convert to upper case once at the start, then you only need to check 'A' to 'Z'.
- A char is a number underneath. 'C' - 'A' is 2, so 'C' - 'A' + 1 is 3.
- Write one tiny method per question you ask about a character: isLetter(c), isVowel(c). Your loops then read like English.
- It is fine to loop over the name three times (vowels, consonants, letter power) if it makes each method simpler. Clarity beats cleverness here.

Level up (optional)

- Find the most repeated letter in the name using an int[26] counter array.
- Print the name with alternating capital letters (LiKe ThIs) using $i \% 2$.

Assignment 4: Mirror Name & Secret Spy Code

Meet your evil twin from the mirror universe.

Concepts you will use

Strings	building a new String with +, toLowerCase(), equals(), substring(), Character.toUpperCase()
Loops	for loop running backwards (i--) and a for loop stepping by 2 (i += 2)
Decisions	if-else on a boolean (isPalindrome) computed once and given a name
Functions	reverse, takeEverySecondLetter, capitalizeFirstLetter, readFirstName, containsOnlyLetters
Data types	double arithmetic with a named constant for the 'seconds of your life' line

The task

Read the user's first name and print it reversed. If the reversed name is identical to the original (a palindrome, like Anna or Hannah), announce a PALINDROME ALERT. Otherwise present the reversed name, with its first letter capitalised, as the user's evil twin from the mirror universe.

Then generate a 'spy code name' by taking every second letter of the name (positions 0, 2, 4 ...) and printing it in capitals. Finish with the number of letters and how many seconds of their life it takes to type (0.3 seconds per letter).

Input rules: a single word of letters only — no spaces, digits or symbols. Re-ask on anything else.

Sample run

(What the user types is shown after the prompt. Your jokes may differ; the numbers must not.)

Sample 1

```
Enter your first name: Ra hul
Please enter letters only - one word, no spaces, digits or symbols.
Enter your first name: Rahul

Your name in the mirror: luhar
Not a palindrome. Your evil twin from the mirror universe is called: Luhar
Your secret spy code name: RHL
Your name has 5 letters. Typing it took 1.5 seconds of your life.
```

Sample 2

```
Enter your first name: Anna

Your name in the mirror: anna
PALINDROME ALERT! Your name reads the same both ways.
You are the same person before and after exams. Rare.
Your secret spy code name: AN
Your name has 4 letters. Typing it took 1.2 seconds of your life.
```

Hints

- To reverse, start a loop at text.length() - 1 and go down to 0, appending each charAt(i) to an empty String.
- Compare Strings with equals(), never with ==.

- Compute boolean `isPalindrome = lowerCaseName.equals(reversedName)` once and use the name; do not repeat the comparison inside the `if`.
- `capitalizeFirstLetter` should guard against an empty `String` before calling `charAt(0)`.

Level up (optional)

- Ask for the full name and reverse the order of the words (Rahul Kumar → Kumar Rahul) without using `split()`.
- Check whether the name is a palindrome when spaces and case are ignored ('Nurses run').

Assignment 5: FLAMES Calculator

The last-bench relationship algorithm, finally in Java.

Concepts you will use

Arrays	char[] from toCharArray(); marking used letters with a named constant CANCELLED_LETTER_MARK
Nested loops	for-inside-for to cancel matching letters one pair at a time, with break
Loops	while loop that shrinks the word FLAMES until one letter is left
Decisions	switch on the surviving letter; validation ifs
Strings	toLowerCase(), replace(), substring() via a removeCharacterAt helper

The task

Implement the school-bench game FLAMES. Read two names. Cancel out common letters one pair at a time (if 'Rahul' and 'Priya' share one 'r' and one 'a', exactly one 'r' and one 'a' are removed from each). Count the letters that remain in both names together.

Now count around the word FLAMES using that number, removing the letter you land on, and continue counting from the next letter with the shortened word. Repeat until a single letter remains: F = Friends, L = Love, A = Affection, M = Marriage, E = Enemies, S = Siblings. Print the result with a suitably dramatic message.

Input rules: both names must be non-empty and contain only letters and spaces. Spaces are removed and case is ignored before the game starts.

Step by step

- readName + normalise: lower-case the name and remove spaces.
- countLettersAfterCancelling: for every letter in name 1, look for the same letter in name 2 that is not yet cancelled; if found, mark both and break out of the inner loop. Then count the active letters in both arrays.
- findSurvivingLetter(count): positionToRemove = (startPosition + count - 1) % remainingWord.length(); remove that character; the next round starts at that position; repeat while the word is longer than 1.
- getVerdict(letter): switch that returns the message.

Sample run

(What the user types is shown after the prompt. Your jokes may differ; the numbers must not.)

Sample 1

```
Your name: Rahul
Their name: Priya

Letters left after cancelling: 6
Result: MARRIAGE. Start collecting Aadhaar photocopies.
```

Sample 2

```
Your name: Sanjay
Their name: Divya

Letters left after cancelling: 7
Result: ENEMIES. Fight over the last gulab jamun forever.
```

Hints

- The break inside the inner loop is essential — without it, one 'a' in the first name would cancel every 'a' in the second.
- Give the cancelled-letter marker a name (CANCELLED_LETTER_MARK = '*') so nobody has to guess what '*' means.
- Write removeCharacterAt(text, position) as its own method: substring(0, position) + substring(position + 1).
- Trace it on paper for count = 6: FLAMES → remove S → FLAME → remove F → LAME → remove A → LME → remove L → ME → remove E → M.

Level up (optional)

- Print a 'love percentage' too: (count * 17) % 101.
- Run it for a list of five names stored in a String[] against one name, in a loop.

Assignment 6: Zodiac Roast Machine

Your star sign, but honest.

Concepts you will use

Arrays	parallel arrays SIGN_NAMES, LAST_DAY_OF_SIGN_IN_MONTH, ROASTS — same index means same sign
Decisions	if to decide whether the cut-off day has passed; validation loops
Operators	% NUMBER_OF_SIGNS to wrap December's late dates back to Capricorn
Functions	findSignIndex(day, month) returns an index; readNumberInRange / isWholeNumber for input
Constants	static final arrays and numbers with descriptive names

The task

Read a month and a day. Find the user's (Western) zodiac sign using lookup tables rather than twelve separate if statements: store the twelve sign names in one array and, in another array, the last day of each month on which the previous sign still applies (19 Jan is still Capricorn; 20 Jan is Aquarius).

Print the sign in capitals, a one-line roast for that sign, and a 'lucky snack' chosen from a small array using the sign's index.

Input rules: read the month first (1-12), then limit the day to the real length of that month (February allows 29 so leap-day birthdays work). Anything else is re-asked.

Sample run

(What the user types is shown after the prompt. Your jokes may differ; the numbers must not.)

Sample 1

```
Enter your birth month (1-12): 8
Enter your birth day (1-31): 32
Please enter a value between 1 and 31.
Enter your birth day (1-31): 15

Your zodiac sign: LEO
Roast: Enters the room like it is a stage. The room was a lift.
Lucky snack: momos
```

Sample 2

```
Enter your birth month (1-12): 12
Enter your birth day (1-31): 25

Your zodiac sign: CAPRICORN
Roast: Works hard, sleeps harder. Has a 5-year plan and a 5-minute attention span.
Lucky snack: samosa
```

Hints

- Order the signs array starting with Capricorn (the sign in force on 1 January). Then month - 1 is the index of the sign at the start of that month.
- If day > LAST_DAY_OF_SIGN_IN_MONTH[month - 1], move to the next index. For December that gives 12, so apply % 12 to come back to 0 (Capricorn).

- Validate before you look up — an out-of-range index will crash with `ArrayIndexOutOfBoundsException`.
- Write a comment above each lookup table explaining what the index means. Tables without comments are puzzles.

Level up (optional)

- Add a fourth parallel array of 'ideal careers' (all of them slightly insulting).
- Accept the whole date and also print the Chinese zodiac animal using `year % 12` and another array.

Assignment 7: Which Day Were You Born?

Monday blues since Day 1. Literally.

Concepts you will use

Integer maths	Zeller's formula — only +, *, / and %, broken into named terms
Decisions	if to treat January and February as months 13 and 14 of the previous year; switch on the weekday using named constants
Arrays	WEEKDAY_NAMES in the order the formula produces; DAYS_IN_MONTH for validation
Functions	calculateWeekday(d, m, y) — called twice; isLeapYear; getDaysInMonth; getWeekdayJoke

The task

Read a full date of birth and print which day of the week the user was born on, using Zeller's congruence (given below) — no Date or Calendar classes. Then print a joke about being a Monday baby, a Sunday baby, and so on. Finally, call the same function again to tell the user which weekday their birthday falls on this year.

Zeller's formula: if month is January or February, add 12 to the month and subtract 1 from the year. Let $k = \text{year} \% 100$ and $j = \text{year} / 100$. Then $h = (\text{day} + 13 * (\text{month} + 1) / 5 + k + k / 4 + j / 4 + 5 * j) \% 7$, where 0 = Saturday, 1 = Sunday, 2 = Monday ... 6 = Friday. All divisions are integer divisions.

Input rules: year 1900-2026, month 1-12, and a day that really exists in that month and year (29 February 1900 must be rejected — 1900 was not a leap year).

Sample run

(What the user types is shown after the prompt. Your jokes may differ; the numbers must not.)

Sample 1

```
Enter your birth year (1900-2026): 1947
Enter your birth month (1-12): 8
Enter your birth day (1-31): 15

You were born on a Friday!
Friday baby: the party started when you arrived. Still going.
This year your birthday falls on a Saturday.
```

Sample 2

```
Enter your birth year (1900-2026): 1900
Enter your birth month (1-12): 2
Enter your birth day (1-28): 29
Please enter a value between 1 and 28.
Enter your birth day (1-28): 28

You were born on a Wednesday!
Wednesday baby: mid-week, mid-bench, mid-attendance. Balanced.
This year your birthday falls on a Saturday.
```

Hints

- Integer division is your friend here: $13 * (\text{month} + 1) / 5$ must be computed with ints, not doubles.

- Split the formula into named parts — monthTerm, yearTerm, centuryTerm — so a reader can check each piece against the textbook.
- Put the names in an array in Zeller's order (Saturday first), then WEEKDAY_NAMES[h] is the answer with no if statements.
- Check against a date you know: 15 August 1947 was a Friday; 26 January 1950 was a Thursday.

Level up (optional)

- Print on which weekday the user's 18th, 25th and 60th birthdays fall, using a for loop.
- Count how many times their birthday has fallen on a Sunday so far.

Assignment 8: Which Chai Are You? (Personality Quiz)

Filter coffee, masala chai, cutting chai or cold coffee — the code decides.

Concepts you will use

Arrays	QUESTIONS array — the quiz lives in data, not in code
do-while	ask a question at least once and keep re-asking while the answer is not 1-4 (including non-numeric input)
for loop	loop over the questions array, accumulating the total
Decisions	if for validation, switch on the score category using named constants
Functions	askQuestion(scanner, number, text) returns a validated int; calculateCategory; getChaiDescription

The task

Write a five-question personality quiz. Store the questions (each with four options) in a String array. For each question, print it and read an answer from 1 to 4; if the user types anything else — a number out of range, letters, or nothing — complain and ask the same question again until a valid answer is given. This is a do-while job.

Add up the five answers (the total is between 5 and 20) and map the total to one of four results: FILTER COFFEE (strong and punctual), MASALA CHAI (warm and spicy), CUTTING CHAI (small effort, maximum vibe) or COLD COFFEE (mostly ice, please wake up). Print the score and the result with a suitably cheeky description.

The score-to-category calculation must be derived from the number of questions and options (constants), not hard-coded, so that adding a sixth question still works.

Sample run

(What the user types is shown after the prompt. Your jokes may differ; the numbers must not.)

```
=== WHICH CHAI ARE YOU? ===

Q1. You wake up at: 1) 5 AM  2) 8 AM  3) 11 AM  4) 'morning' is a concept
Your answer (1-4): 2

Q2. Group project: 1) You lead  2) You help  3) You add your name  4) You ask 'which
project?'
Your answer (1-4): 7
Bro, only 1 to 4. Try again.

Q2. Group project: 1) You lead  2) You help  3) You add your name  4) You ask 'which
project?'
Your answer (1-4): x
Bro, only 1 to 4. Try again.

Q2. Group project: 1) You lead  2) You help  3) You add your name  4) You ask 'which
project?'
Your answer (1-4): 3

Q3. Exam tomorrow: 1) Revised twice  2) Started today  3) Starting now  4) What exam?
Your answer (1-4): 1

Q4. Weekend plan: 1) Gym  2) Movie  3) Sleep  4) Same as weekday
```

Your answer (1-4): 4

Q5. Your phone battery right now: 1) 100% 2) 60% 3) 20% 4) Charger is my best friend

Your answer (1-4): 2

Your score: 12/20

You are MASALA CHAI. Warm, popular, a bit spicy in arguments.

Hints

- `do { ... } while (!isValidAnswer);` — the body runs first, then the condition is checked. That is exactly what 'ask at least once' means.
- Read the answer as a line of text; if it is not all digits, treat it as 0 (invalid) rather than letting Scanner throw an exception.
- $\text{Category} = (\text{total} - \text{minimumTotal}) / \text{bucketSize}$ where $\text{bucketSize} = (\text{number of different totals}) / 4$. Check the boundaries: $5 \rightarrow 0$, $8 \rightarrow 0$, $9 \rightarrow 1$, $20 \rightarrow 3$.
- Pass the Scanner into the function instead of creating a new one each time.

Level up (optional)

- Store the four results in a `String[]` too, so the switch disappears entirely.
- Add a 'play again?' loop around the whole quiz (another do-while).

Assignment 9: Guess the Number (With Attitude)

Seven attempts. The computer will not be kind.

Concepts you will use

Random numbers	pickRandomNumber(min, max) wraps Math.random() so the cast and +1 are explained once
while loop	keep playing while !hasGuessedCorrectly AND attemptsUsed < MAX_ATTEMPTS
Arrays	TOO_LOW_TAUNTS and TOO_HIGH_TAUNTS, chosen randomly with the same helper
Decisions	if / else-if for higher-lower; boolean flag for the outcome; if-ladder for the rank
Functions	pickRandomNumber, pickRandomLine, readNumberInRange, getRank

The task

The computer picks a secret number from 1 to 100. The user has up to seven attempts to guess it. After each wrong guess, print a random taunt from an array of 'too low' lines or an array of 'too high' lines ('Too low. Like your attendance.').

When the user wins, print the number of attempts and a rank: 3 or fewer attempts is MIND READER, up to 5 is SOLID, otherwise PASSED (barely). If all seven attempts are used, reveal the number and award the rank BACKLOG.

Input rules: a guess must be a whole number from 1 to 100. Anything else is rejected with a message and does not use up an attempt.

Sample run

(What the user types is shown after the prompt. Your jokes may differ; the numbers must not.)

```
I am thinking of a number between 1 and 100.
You have 7 attempts. Even the WiFi guy needs fewer.

Attempt 1 - your guess: 500
Stay between 1 and 100. That guess does not count.
Attempt 1 - your guess: 50
Too high. Like the canteen prices.
Attempt 2 - your guess: 25
Too low. Like your attendance.
Attempt 3 - your guess: 37
Lower! Come down to reality.
Attempt 4 - your guess: 31
Nope, go up. The number is not in the basement.
Attempt 5 - your guess: 34

Correct! The number was 34. Got it in 5 attempts.
Rank: SOLID. Binary search is proud of you.
```

Hints

- Math.random() returns a double from 0.0 up to (but not including) 1.0. Put the 'multiply, cast, add' logic into one method pickRandomNumber(min, max) and reuse it for the taunts too.
- Use a boolean flag hasGuessedCorrectly instead of comparing guess to the secret in both the loop condition and the final if.

- Because invalid guesses are handled inside `readNumberInRange`, the game loop only ever sees valid numbers and only counts those as attempts.

Level up (optional)

- Let the user pick the range (1 to N) and compute the fair number of attempts as the number of times you can halve N.
- Keep a best-score across rounds using a do-while 'play again' loop.

Assignment 10: Password Roaster

Your grandmother's phone lock is stronger than this.

Concepts you will use

Characters	range checks wrapped in <code>isDigit</code> , <code>isUpperCaseLetter</code> , <code>isLowerCaseLetter</code>
boolean flags	<code>hasDigit</code> , <code>hasUpperCase</code> , <code>hasLowerCase</code> , <code>hasSymbol</code> , <code>isLongEnough</code> — set once, used in three places
for loop	one pass over the password sets all four flags; a loop over <code>FAMOUS_BAD_PASSWORDS</code>
Decisions	score counting with independent ifs; an ordered if-ladder for the verdict; <code>contains()</code> and <code>equals()</code>
Functions	<code>calculateScore</code> , <code>isFamousBadPassword</code> , <code>containsName</code> , <code>getVerdict</code> , <code>printTips</code>

The task

Read the user's name and a password. In a single loop over the password, detect whether it contains a digit, an upper-case letter, a lower-case letter and a symbol (anything else). Award one point for each of those plus one point for a length of 8 or more, giving a score out of 5.

Then roast the password: if it contains the user's own name (ignoring case), say so; if it is one of the famous bad passwords (123456, password, 12345678, qwerty ...), say so; otherwise roast or praise according to the score. Finish with practical tips for whatever is missing ('add a number — no, not your birth year').

Input rules: neither the name nor the password may be empty. Keep the list of famous bad passwords in an array so it can grow.

Sample run

(What the user types is shown after the prompt. Your jokes may differ; the numbers must not.)

Sample 1

```
Enter your name: Rahul
Enter a password (don't worry, we won't tell anyone): rahul123

Password strength: 3/5
Roast: Your name is IN your password. Hackers thank you for your service.
Tip: add a symbol like @ or #. Hashtags finally useful.
```

Sample 2

```
Enter your name: Amit
Enter a password (don't worry, we won't tell anyone):
This cannot be empty. Please try again.
Enter a password (don't worry, we won't tell anyone): 123456

Password strength: 1/5
Roast: This password is on every hacker's first-try list. Bold move.
Tip: add a symbol like @ or #. Hashtags finally useful.
Tip: make it at least 8 characters. Short passwords, short careers.
```

Hints

- Write `isDigit(char)` yourself rather than using `Character.isDigit` — the point is to practise the comparison.
- The `else` at the end of the `if / else-if` chain inside the loop is what catches symbols: not a digit, not upper, not lower, so it must be something else.
- Put the famous bad passwords in a `String[]` and loop over it with `equals()`; never chain five `==` comparisons.
- Order the verdict checks from most specific to least specific: name-in-password, famous password, then score.

Level up (optional)

- Add a point if no character repeats three times in a row (aaa) — needs a loop that looks at `i`, `i+1` and `i+2`.
- Generate a suggested strong password by mixing the name with random digits and symbols.